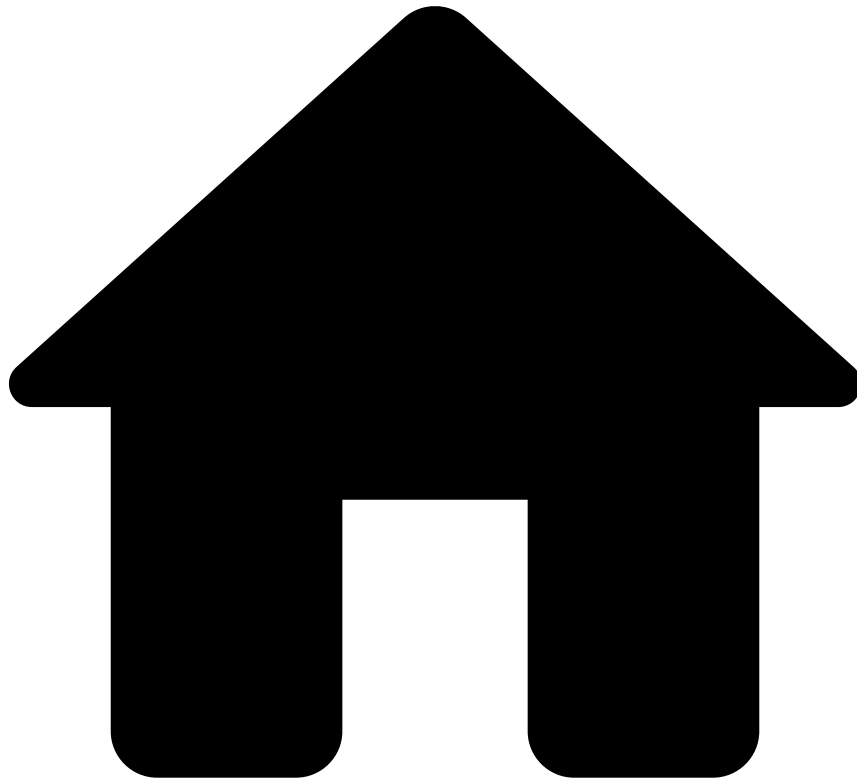


•



- [Component Encryption](#)
- Creating Certificates

On this page
Was this page helpful? 

Creating Certificates

Pulse includes scripts to help creating keystores and truststores for component encryption.

Both of these files contain certificates that are used for authentication. This guide explains how to use the scripts to configure [Component Encryption with Server Authentication](#).

The scripts generate:

- Server keystores with a private key and a certificate chain with two entries:
 - Server certificate signed by a Certificate Authority (CA).
 - Public certificate issued by the CA (i.e CA certificate).
- Client truststore with the CA certificate.

The certificates are generated both in JKS which is typically required by Java-based components (e.g. application engine, Pulse server) and PEM formats required by some external components (e.g. Rabbit).

Selecting server authentication approach

This section explains component encryption configured with server authentication. For each Feedzai's customers, the authentication approach is selected on a case-by-case basis, and normally fall under one of these options:

- **Self-signed certificates**

Requires generating self-signed certificates and configuring all Pulse components to trust them. This approach assumes that the person generating the certificates has also the role of internal CA. As an example, this could be configured by a Feedzai employee from the delivery team (i.e. referred to as Feedzai in this guide). A typical scenario for this approach is to use it in a test environment.



info

Pulse distributions include a `bin/create-keystore.sh` script to generate self-signed certificates. This could also be used on a configuration where self-signed certificates are used instead of certificate chains. This option is simpler when only one component is secured (i.e. typically jetty).

- **Certificates signed by an internal CA**

Requires receiving certificates signed by an internal CA and configuring all the Pulse components to trust this CA. In this approach, the internal CA is generally handled by a separate department belonging to a Feedzai's customer.

A typical scenario to use this approach is a production environment. Typically, Feedzai's customers have an internal CA that might be handled by a department of security (i.e. referred to as client security team in this guide).

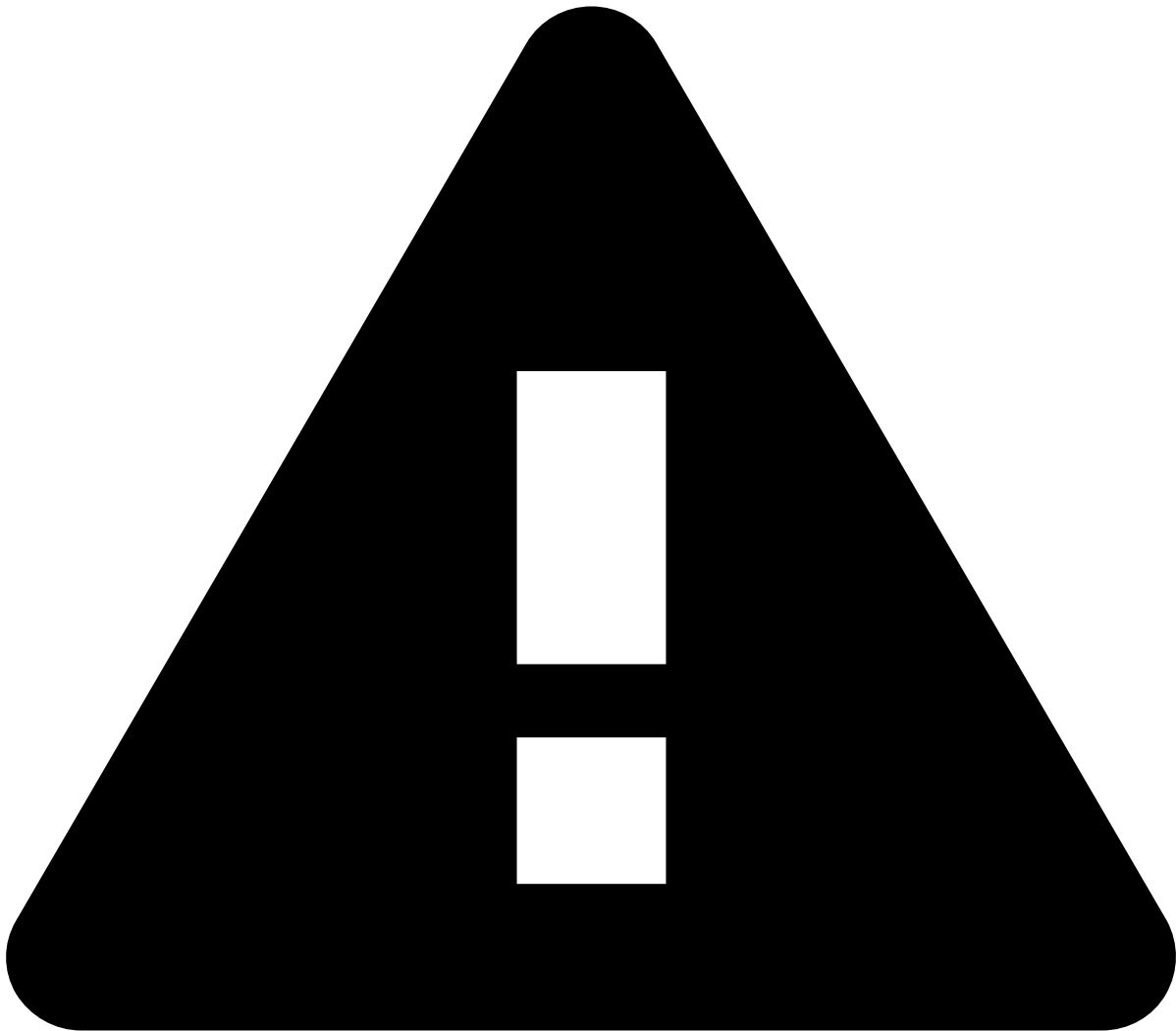
Handling the configuration procedure📄

The configuration procedure can be divided in 3 main steps:

1. Configure a CA that is used to issue a CA certificate as well as to sign the server certificate. Generally, this is only required for the self-signed certificate approach.
2. Configure a truststore with a CA certificate that can be then placed in Pulse components acting as clients.
3. Configure a keystore that can then be placed in Pulse components acting as servers. The keystore contains a certificate chain (server certificate signed by the CA with a digital signature, and the CA certificate) and a private key.

The following diagram illustrate the 3 main steps, including the main differences between a testing and a production environment.

Configuring encryption with server authentication^â



warning

The scripts below generate working files with sensitive information such as private keys and passwords. The files are stored in a directory named "secret" that is created by the scripts, but are not required by Pulse or any component.

The scripts should not be executed in a running pulse machine. The scripts only require the openssl and keytool tools to be available on the machine that is used to execute them.

The scripts are located in the folder *certificates* inside the pulse folder.

The scripts can be open using a text editor and they contain configuration instructions. The recommended order to configure the scripts is the following:

1. Open and execute commands in **create-ca.sh**.

This script must be executed first because it contains the CA Certificate that will be used in commands by the other scripts.

2. Open and execute commands in **create-truststore.sh**.

Generates a truststore in a JKS file with the CA Certificate (file truststore.jks). The password to access this file is stored in the secret/truststore.pwd file.

3. Open and execute commands in **create-keystore.sh**.

Generates a keystore with a private key, a certificate signed by the CA, and the CA certificate (file keystore.jks). The keystore password and the private key password are stored in the secret/keystore.pwd and secret/privatekey.pwd files, respectively.

Alternatively, the execution order of 2 and 3 can be switched.

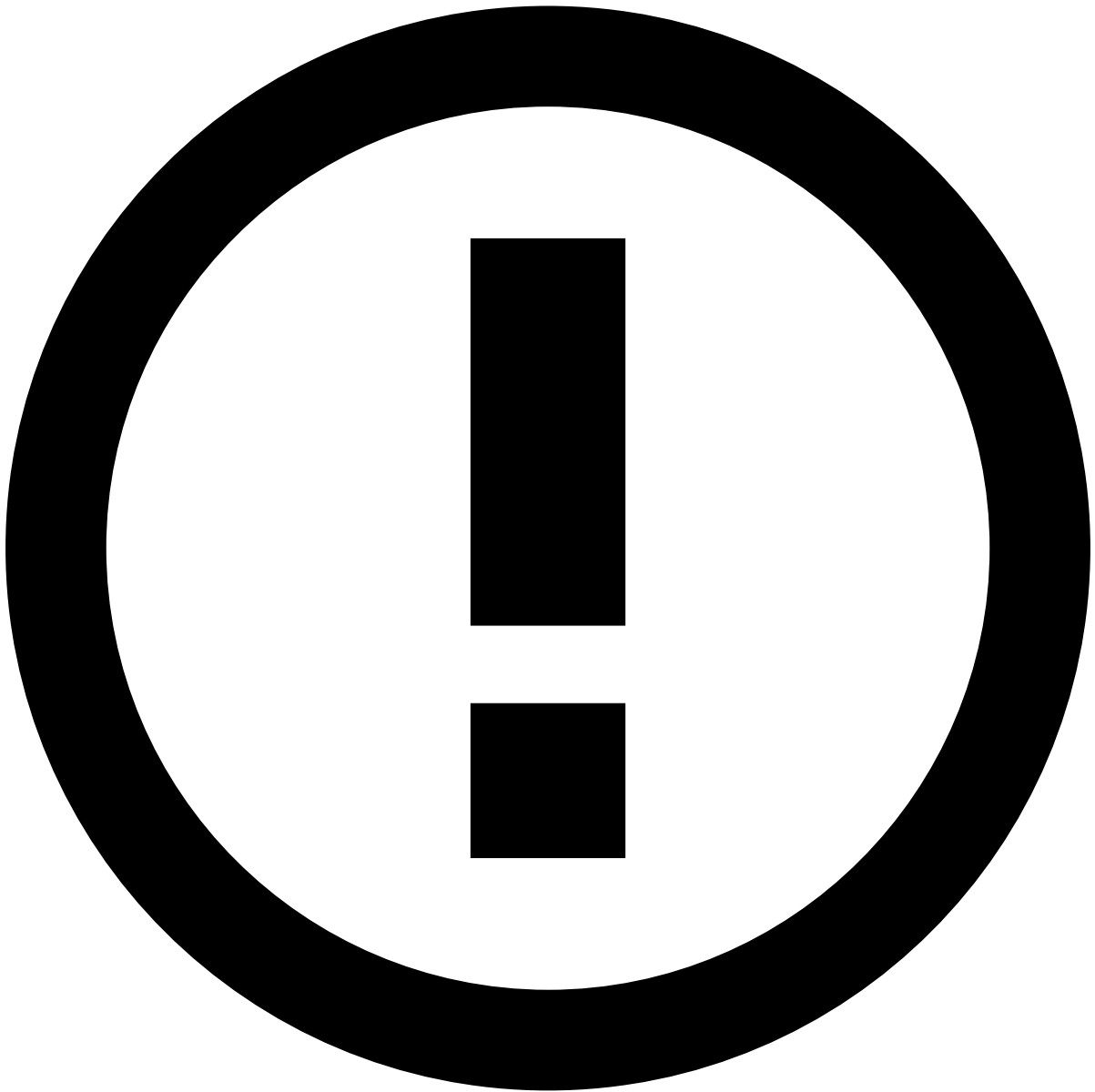
The size and validation of the private keys are set when the scripts are configured. The recommended values are 2048 byte and 10 years, respectively.

Testing environment

This section guides you through the configuration of the scripts in a typical testing environment scenario. In addition to configure truststores and keystores in Pulse components, the Feedzai team plays the role of CA.

To configure component encryption with server authentication execute the following actions:

Run `create-ca.sh` to generate the CAâ

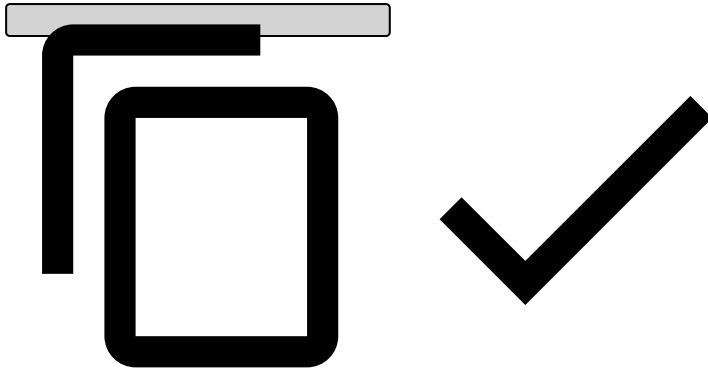


Do you have CA configured?

If you have already a CA configured, skip this step and jump to the subsection [Run create-truststore.sh to generate the truststore](#).

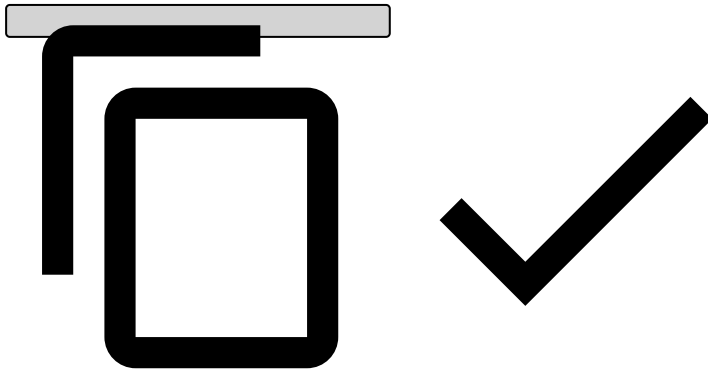
1. Navigate to the `pulse/certificates` folder and open the script `create-ca.sh` with a text editor.
2. Follow the instructions described in the script. The main actions are the following:
 1. Create a directory with the name `secret` (i.e. `certificates/secret`) to store your CA password. If you have already this directory, you can skip this step and continue to the next step.
 2. Generate a file with a CA password using the following command:

```
$ apg -n 1 -m 32 > secret/ca.pwd  
$ CERT_PWD=`cat secret/ca.pwd`
```



3. Generate the CA Certificate file and the CA private key by executing the following (file output `rootCA.crt`):

```
$ openssl req -new -x509 -nodes -newkey rsa:$KEY_SIZE -subj /CN=rootCA/OU=Pulse/O=F  
eedzai/C=PT/ -keyout $CA_KEY_FILE -out $CA_CERT_FILE -days $CERT_DAYS -passin pass:$  
CERT_PWD
```

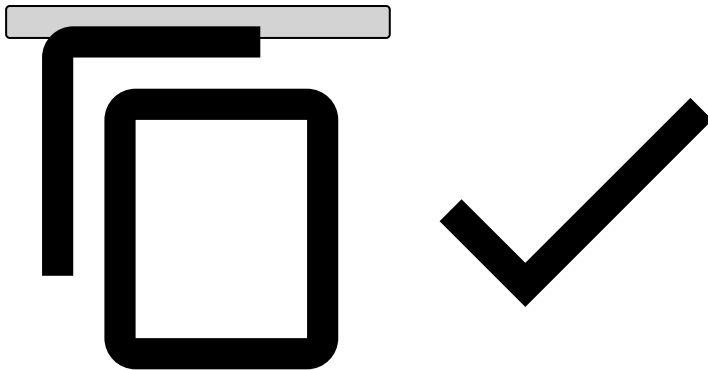


where the variables are replaced by the following (recommended naming):

Variables	Values
<code>\$CA_CERT_FILE</code>	<code>rootCA.crt</code>
<code>\$CA_KEY_FILE</code>	<code>secret/rootCA.key</code>
<code>\$CERT_NAME</code>	<code>rootCA</code>
<code>\$CERT_DAYS</code>	<code>3650</code>
<code>\$KEY_SIZE</code>	<code>2048</code>
<code>\$CERT_PWD</code>	Defined in step 2(b)

As a verification step you can check the content of the CA Certificate with the following command:

```
$ openssl x509 -in $CA_CERT_FILE -text -noout
```

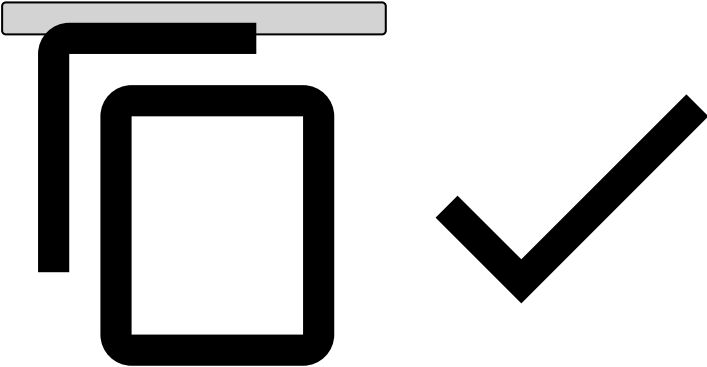


Run create-truststore.sh to generate the truststore

- 1. Navigate to the pulse/certificates folder and open the script create-truststore.sh with a text editor.
- 2. Follow the instructions described in the script. The main actions are the following:

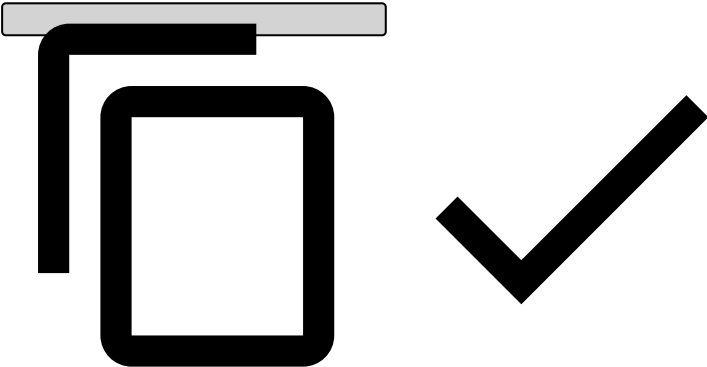
- 1. Generate a file with the truststore password by executing the following:

```
$ apg -n 1 -m 32 > secret/truststore.pwd
$ TRUST_STORE_PWD=`cat secret/truststore.pwd`
```



- 2. Generate the truststore file in a JKS format using the CA Certificate that is trusted by all components acting as clients (i.e. certificate rootCA.crt generated in the subsection [Run create-ca.sh to generate the CA](#)). To do so, execute the following command (file output trustedca.jks):

```
$ keytool -importcert -keystore $TRUST_STORE_FILE -alias rootCa -file $CA_CERT_FILE
-noprompt -storepass $TRUST_STORE_PWD
```

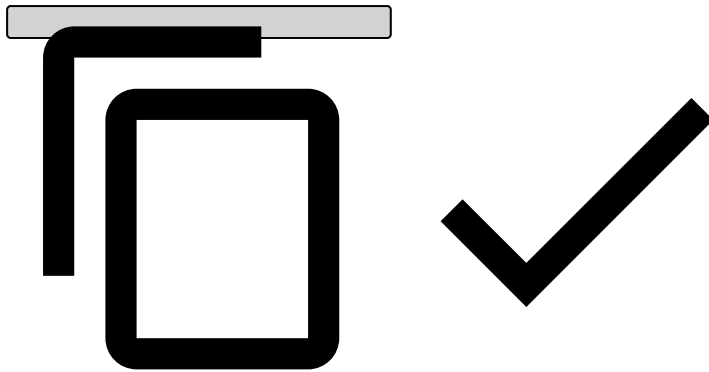


where the variables are replaced by the following (recommended naming):

Variables	Values
\$TRUST_STORE_FILE	trustedca.jks
\$CA_CERT_FILE	rootCA.crt
\$TRUST_STORE_PWD	Defined in step 2(a)

The password defined for the variable \$TRUST_STORE_PWD is requested when opening the file. This password avoid unapproved file editing, for example, changing the certificates within the file. As a verification step, check the file trustedca.jks by executing the following command:

```
$ keytool -list -keystore $TRUST_STORE_FILE -storepass $TRUST_STORE_PWD
```

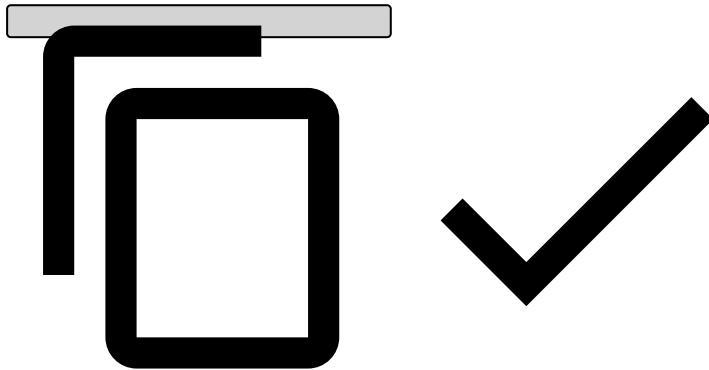


Run `create-keystore.sh` to generate the keystore[🔗](#)

1. Navigate to the `pulse/certificates/` folder and open the script `create-keystore.sh` with a text editor.
2. Follow the instructions described in the script. The main actions are the following:

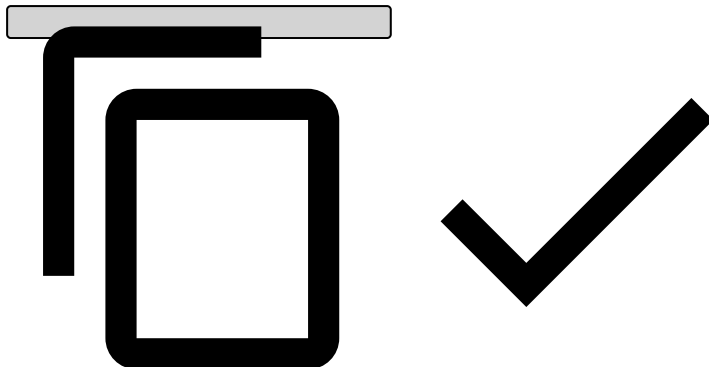
1. Generate a file with the keystore password by executing the following:

```
$ apg -n 1 -m 32 > secret/keystore.pwd  
$ SERVER_STORE_PWD=`cat secret/keystore.pwd`
```



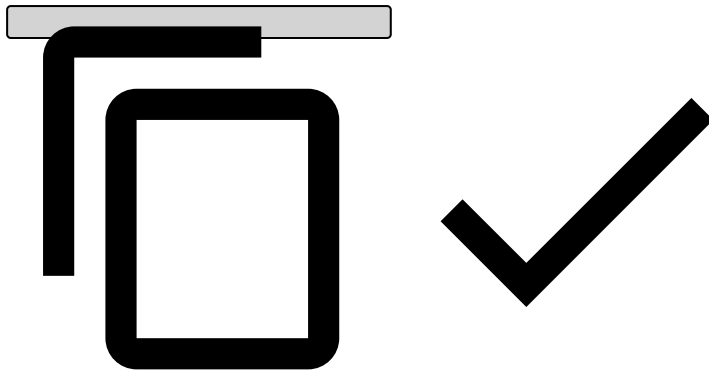
2. Generate a file with the private key password by executing the following:

```
$ apg -n 1 -m 32 > secret/privatekey.pwd  
$ SERVER_KEY_PWD=`cat secret/privatekey.pwd`
```



3. Generate a private key with the following command (file output `key.pem`):

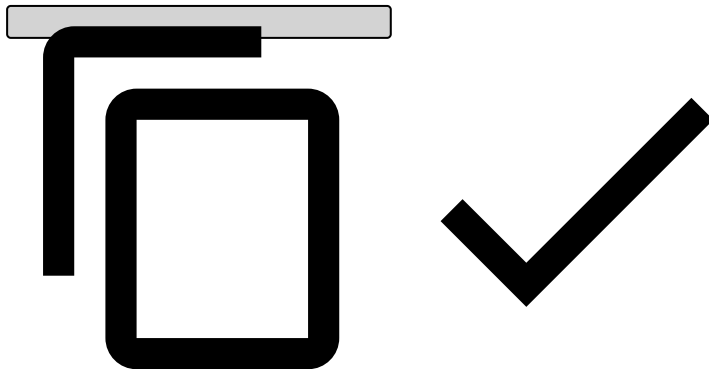
```
$ openssl genrsa -out key.pem $KEY_SIZE
```



where the variable `$KEY_SIZE` can be replaced by 2048.

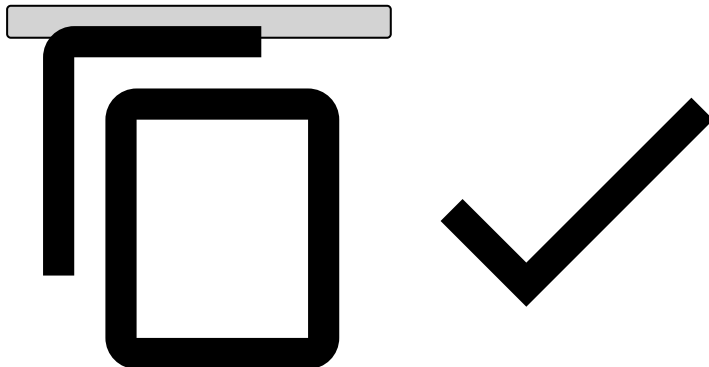
4. Generate the certificate signing request (csr) with the following command (file output `req.pem`):

```
$ openssl req -new -key key.pem -out req.pem -outform PEM -subj /CN=pulse/0=server/ -nodes
```



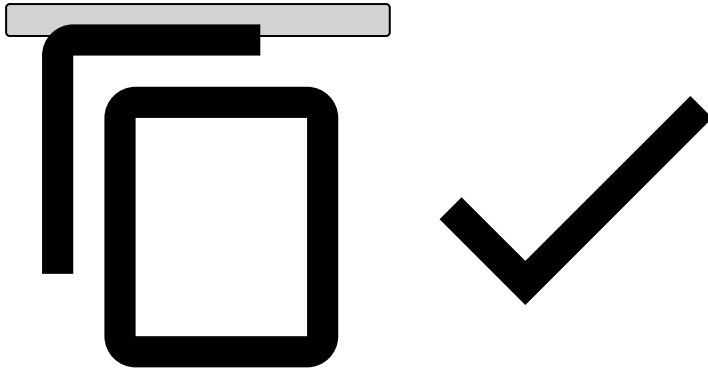
Then run the following command to visualize the certificate signing request and verify that it has no issuer yet:

```
$ openssl req -text -in req.pem
```



5. Performing the CA role, sign the certificate signing request using the certificate `rootCA.crt` and the root private key with the following command (file output `req.signed.pem`):

```
$ openssl x509 -req -CA $CA_CERT_FILE -CAkey $CA_KEY_FILE -in req.pem -out req.signed.pem -days $CERT_DAYS -CAcreateserial -passin pass:$CERT_PWD
```



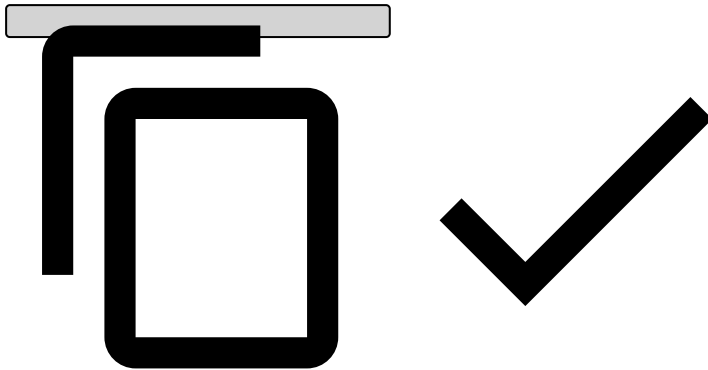
where the variables are replaced by the following (recommended naming):

Variables	Values
<code>\$CA_CERT_FILE</code>	rootCA.crt
<code>\$CA_KEY_FILE</code>	secret/rootCA.key
<code>\$CERT_DAYS</code>	3650
<code>\$CERT_PWD</code>	Choose a password

The password chosen for the variable `$CERT_PWD` is used to generate the digital signature that is added to the certificate.

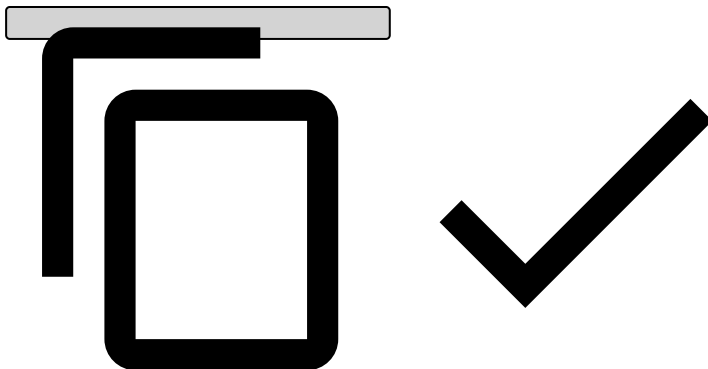
As a verification step, run the following command and verify that the certificate has already an issuer (i.e. rootCA), in opposition to the verification that was done in Step 2(d).

```
$ openssl x509 -in req.signed.pem -text -noout
```



6. Create a package in a pkcs12 format containing the certificate chain (self-signed certificate and the CA certificate) and the server private key with the following command (file output `server.pkcs12`):

```
$ openssl pkcs12 -export -out keystore.pkcs12 -in req.signed.pem -inkey key.pem -certfile $CA_CERT_FILE -passout pass:$SERVER_STORE_PWD
```



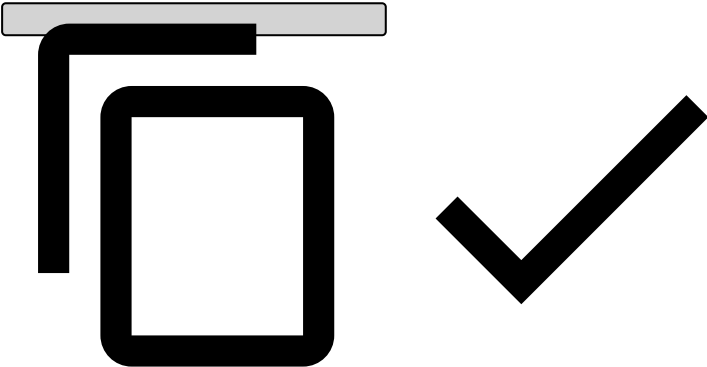
where the variables are replaced by the following (recommended naming):

Variables	Values
\$CA_CERT_FILE	rootCA.crt
\$SERVER_STORE_PWD	Defined in Step 2(a)

The password defined for the variable `$SERVER_STORE_PWD` is used in the next step to convert the file to a JKS format.

7. Convert the pkcs12 file to JKS format using the following command (file output `keystore.jks`):

```
$ keytool -importkeystore -srckeystore keystore.pkcs12 -srcstoretype PKCS12 -srcstorepass $SERVER_STORE_PWD -destkeystore $SERVER_KEY_STORE_FILE -deststoretype JKS -noprompt --deststorepass $SERVER_STORE_PWD -destkeypass $SERVER_KEY_PWD
```

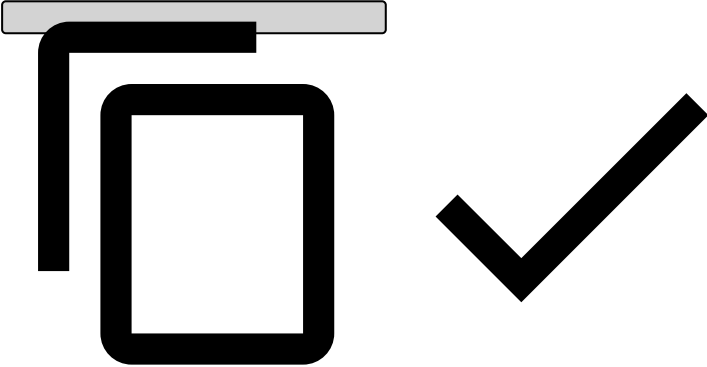


where the variables are replaced by the following (recommended naming):

Variables	Values
\$SERVER_STORE_PWD	The password defined in Step 2(d), i.e. ServerStorePassValue
\$SERVER_KEY_STORE_FILE	keystore.jks
\$SERVER_STORE_PWD	Defined in Step 2(a)
\$SERVER_KEY_PWD	Defined in Step 2(b)

As a verification step run the following command:

```
$ keytool -list -v -keystore $SERVER_KEY_STORE_FILE -storepass $SERVER_STORE_PWD -keypass $SERVER_KEY_PWD
```

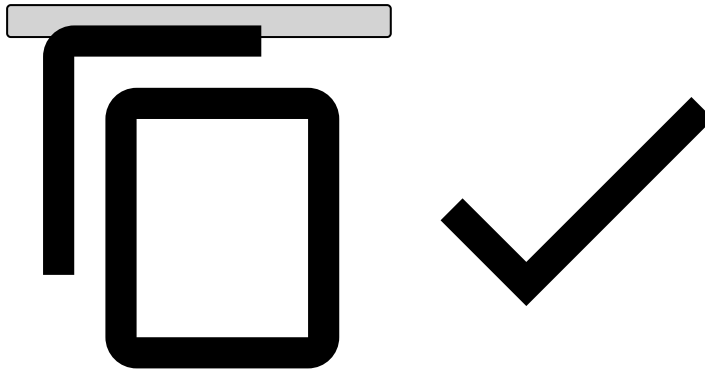


The output gives you information about:

- Number of keystores: "Your keystore contains N entry"
- Self-signed certificate: The first certificate fingerprints
- CA certificate: The second certificate fingerprints

8. Clean up temp files with the following command:

```
$ rm -f key.pem req.pem req.signed.pem
```



Production environment

In a typical production environment, Feedzai has the responsibility to configure truststore and keystore files in Pulse components whereas Feedzai's customer manages the CA.

The procedure to configure the scripts in a production environment is similar to the procedure used in testing environments. Therefore, this section only highlights the differences between both environments and you are referred to section [Testing environment](#) for more details about the full procedure.

Generate a CA

In a scenario of production environment, the script `create-ca.sh` is not needed because the CA is managed by Feedzai's customers.

Run `create-truststore.sh` to generate the truststore

In a testing environment, Feedzai creates a truststore using the CA certificate generated by this team using the script `create-ca.sh`. In a Production environment, Feedzai has to:

1. Contact the client security department that is responsible for managing CA certificates to request a CA certificate file.
2. Generate a truststore using the CA certificate sent by the CA (see image below).

For more information about creating a truststore, check the subsection [Run `create-truststore.sh` to generate the truststore](#) under the section [Testing environment](#).

Run `create-keystore.sh` to generate the keystore

In a testing environment, Feedzai team signs the certificate signing request by performing the role of internal CA.

In a Production environment, Feedzai has to:

1. Generate the certificate signing request.
2. Contact the client security department that is responsible for managing the CA certificates and send the certificate signing request file to be signed by the CA.
3. Create a keystore containing the certificate chain (certificate signed by the CA and the CA certificate) and the server private key (see the following image).

For more information about creating a keystore, check the subsection [Run `create-keystore.sh` to generate the keystore](#) under the section [Testing environment](#).

Learn more

Now that you have basic knowledge about the scripts available to generate keystores and truststores, check the page [Configuration Overview](#) to learn more details about configuration for each component in a Fraud Prevention solution.

You can also learn more about the concept of [Component Encryption](#) in Pulse.